

Exercise 3

Advanced Methods for Regression and Classification

Kyzer Gerez

29.10.2025

```
load("building.RData")

# split into 2/3 train 1/3 test
smp_size <- floor(2/3 * nrow(df))

# set the seed to make partition reproducible
set.seed(1)
train_ind <- sample(seq_len(nrow(df)), size = smp_size)

train <- df[train_ind, ]
test <- df[-train_ind, ]
```

(1) *Principal component regression (PCR)*:

- (a) Use the training data to fit a PCR model. PCR is implemented in the `library(pls)` as the function `pcr()`, see help. Perform cross-validation using 10 segments (see help of `pcr()`) and scale the data (`scale=TRUE`).

```
library(pls)

model.pcr <- pcr(y ~ .,
  data=train,
  validation="CV",
  segments=10,
  segment.type="random",
  scale=TRUE)

summary(model.pcr)
```

```
## Data:      X dimension: 248 107
## Y dimension: 248 1
## Fit method: svdpc
## Number of components considered: 107
##
## VALIDATION: RMSEP
## Cross-validated using 10 random segments.
##      (Intercept)  1 comps  2 comps  3 comps  4 comps  5 comps  6 comps
## CV           0.8148   0.5859   0.5837   0.5447   0.5232   0.5187   0.4273
## adjCV        0.8148   0.5857   0.5836   0.5457   0.5230   0.5198   0.4254
##      7 comps  8 comps  9 comps 10 comps 11 comps 12 comps 13 comps
## CV           0.4203   0.3830   0.3394   0.3360   0.3343   0.3364   0.3327
## adjCV        0.4195   0.3802   0.3371   0.3341   0.3326   0.3348   0.3310
```

```

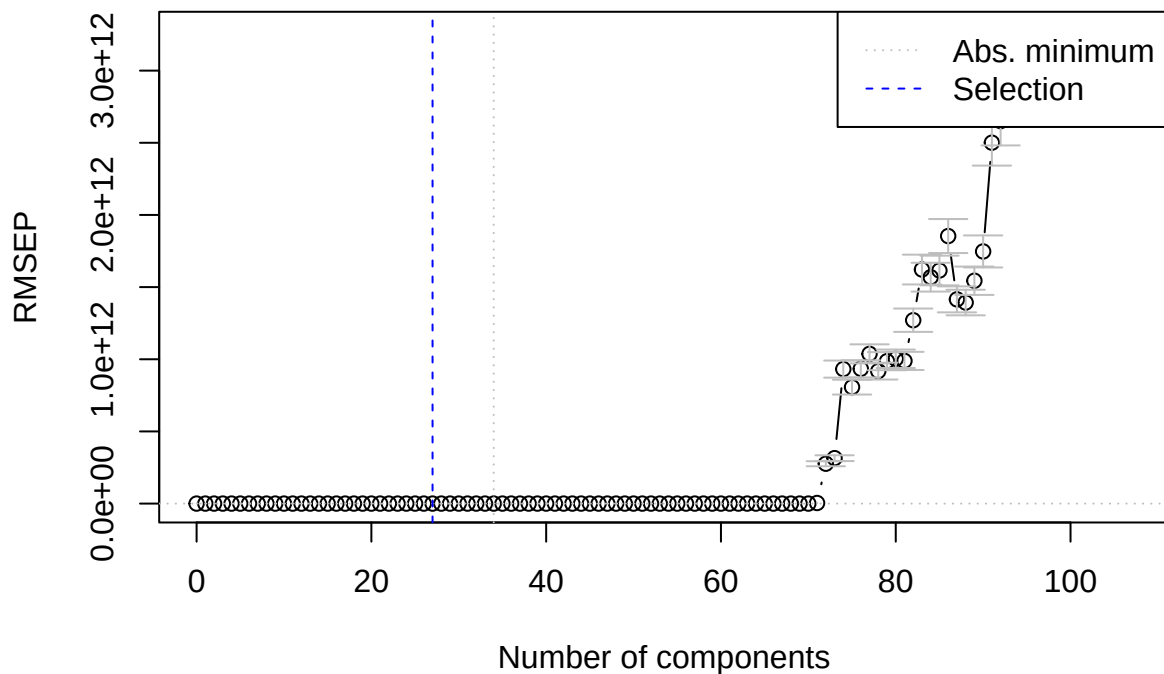
##      14 comps  15 comps  16 comps  17 comps  18 comps  19 comps  20 comps
## CV      0.3323   0.3326   0.3251   0.2852   0.2823   0.2820   0.2793
## adjCV    0.3304   0.3309   0.3244   0.2818   0.2801   0.2802   0.2792
##      21 comps  22 comps  23 comps  24 comps  25 comps  26 comps  27 comps
## CV      0.2706   0.2633   0.2557   0.2567   0.2593   0.2521   0.2433
## adjCV    0.2719   0.2606   0.2539   0.2548   0.2587   0.2480   0.2414
##      28 comps  29 comps  30 comps  31 comps  32 comps  33 comps  34 comps
## CV      0.2447   0.2380   0.2312   0.2290   0.2300   0.2296   0.2288
## adjCV    0.2439   0.2358   0.2293   0.2271   0.2283   0.2276   0.2265
##      35 comps  36 comps  37 comps  38 comps  39 comps  40 comps  41 comps
## CV      0.2294   0.2304   0.2345   0.2307   0.2305   0.2309   0.2324
## adjCV    0.2275   0.2286   0.2333   0.2284   0.2284   0.2288   0.2302
##      42 comps  43 comps  44 comps  45 comps  46 comps  47 comps  48 comps
## CV      0.2338   0.2383   0.2403   0.2410   0.2402   0.2413   0.2435
## adjCV    0.2315   0.2359   0.2378   0.2379   0.2374   0.2386   0.2412
##      49 comps  50 comps  51 comps  52 comps  53 comps  54 comps  55 comps
## CV      0.2458   0.2426   0.2436   0.2426   0.2442   0.2456   0.2470
## adjCV    0.2430   0.2394   0.2404   0.2394   0.2413   0.2421   0.2434
##      56 comps  57 comps  58 comps  59 comps  60 comps  61 comps  62 comps
## CV      0.2516   0.2506   0.2523   0.2600   0.2612   0.2590   0.2608
## adjCV    0.2480   0.2464   0.2483   0.2555   0.2567   0.2543   0.2560
##      63 comps  64 comps  65 comps  66 comps  67 comps  68 comps  69 comps
## CV      0.2571   0.2635   0.2751   0.2897   0.2927   0.2919   0.3708
## adjCV    0.2526   0.2586   0.2693   0.2833   0.2854   0.2851   0.3585
##      70 comps  71 comps  72 comps  73 comps  74 comps  75 comps
## CV      0.3807  3.996e+09  2.761e+11  3.147e+11  9.318e+11  8.068e+11
## adjCV    0.3681  3.797e+09  2.623e+11  2.986e+11  8.841e+11  7.659e+11
##      76 comps  77 comps  78 comps  79 comps  80 comps  81 comps
## CV      9.336e+11  1.038e+12  9.179e+11  9.867e+11  1.003e+12  9.887e+11
## adjCV    8.860e+11  9.844e+11  8.711e+11  9.366e+11  9.522e+11  9.379e+11
##      82 comps  83 comps  84 comps  85 comps  86 comps  87 comps
## CV      1.270e+12  1.622e+12  1.569e+12  1.615e+12  1.854e+12  1.415e+12
## adjCV    1.205e+12  1.539e+12  1.489e+12  1.533e+12  1.760e+12  1.343e+12
##      88 comps  89 comps  90 comps  91 comps  92 comps  93 comps
## CV      1.393e+12  1.544e+12  1.747e+12  2.501e+12  2.650e+12  2.971e+12
## adjCV    1.322e+12  1.466e+12  1.658e+12  2.375e+12  2.517e+12  2.822e+12
##      94 comps  95 comps  96 comps  97 comps  98 comps  99 comps
## CV      3.045e+12  2.877e+12  3.010e+12  2.944e+12  3.139e+12  3.277e+12
## adjCV    2.892e+12  2.733e+12  2.859e+12  2.796e+12  2.981e+12  3.112e+12
##      100 comps 101 comps 102 comps 103 comps 104 comps 105 comps
## CV      3.007e+12  3.131e+12  3.023e+12  3.048e+12  2.983e+12  3.210e+12
## adjCV    2.856e+12  2.974e+12  2.871e+12  2.895e+12  2.833e+12  3.049e+12
##      106 comps 107 comps
## CV      3.226e+12  3.175e+12
## adjCV    3.063e+12  3.016e+12
##
## TRAINING: % variance explained
##      1 comps  2 comps  3 comps  4 comps  5 comps  6 comps  7 comps  8 comps
## X      64.57   71.84   76.85   81.37   84.59   87.46   89.33   90.96
## y      48.60   49.46   56.25   60.41   61.13   77.06   77.80   83.88
##      9 comps 10 comps 11 comps 12 comps 13 comps 14 comps 15 comps
## X      92.51   93.53   94.42   95.27   95.99   96.59   97.06
## y      85.41   85.50   85.57   85.66   86.10   86.32   86.34
##      16 comps 17 comps 18 comps 19 comps 20 comps 21 comps 22 comps

```

```
## X      97.40      97.71      97.97      98.20      98.40      98.58      98.75
## y      87.34      90.14      90.23      90.73      90.73      91.15      92.04
##      23 comps 24 comps 25 comps 26 comps 27 comps 28 comps 29 comps
## X      98.89      99.01      99.11      99.20      99.29      99.36      99.43
## y      92.22      92.36      92.47      93.13      93.22      93.22      93.77
##      30 comps 31 comps 32 comps 33 comps 34 comps 35 comps 36 comps
## X      99.48      99.53      99.58      99.62      99.66      99.70      99.73
## y      93.93      94.06      94.09      94.17      94.21      94.22      94.24
##      37 comps 38 comps 39 comps 40 comps 41 comps 42 comps 43 comps
## X      99.76      99.79      99.81      99.83      99.85      99.87      99.89
## y      94.25      94.45      94.46      94.47      94.48      94.48      94.48
##      44 comps 45 comps 46 comps 47 comps 48 comps 49 comps 50 comps
## X      99.90      99.91      99.93      99.94      99.95      99.95      99.96
## y      94.51      94.58      94.59      94.60      94.60      94.71      94.84
##      51 comps 52 comps 53 comps 54 comps 55 comps 56 comps 57 comps
## X      99.97      99.97      99.97      99.98      99.98      99.99      99.99
## y      94.85      94.87      94.87      94.94      94.97      94.97      95.04
##      58 comps 59 comps 60 comps 61 comps 62 comps 63 comps 64 comps
## X      99.99      99.99      99.99      100.00      100.00      100.00      100.00
## y      95.05      95.05      95.06      95.12      95.12      95.15      95.15
##      65 comps 66 comps 67 comps 68 comps 69 comps 70 comps 71 comps
## X      100.00      100.00      100.00      100.00      100.00      100.00      100.00
## y      95.17      95.17      95.28      95.29      95.31      95.36      95.57
##      72 comps 73 comps 74 comps 75 comps 76 comps 77 comps 78 comps
## X      100.00      100.00      100.00      100.00      100.00      100.00      100.00
## y      95.63      95.63      95.72      95.74      95.74      95.87      95.88
##      79 comps 80 comps 81 comps 82 comps 83 comps 84 comps 85 comps
## X      100.00      100.00      100.00      100.00      100.00      100.00      100.00
## y      95.89      95.89      95.91      95.91      95.93      95.93      95.99
##      86 comps 87 comps 88 comps 89 comps 90 comps 91 comps 92 comps
## X      100.00      100.00      100.00      100.00      100.00      100.00      100.00
## y      96.02      96.05      96.09      96.14      96.19      96.20      96.22
##      93 comps 94 comps 95 comps 96 comps 97 comps 98 comps 99 comps
## X      100.00      100.00      100.00      100.00      100.00      100.00      100.00
## y      96.22      96.22      96.23      96.23      96.27      96.30      96.41
##      100 comps 101 comps 102 comps 103 comps 104 comps 105 comps 106 comps
## X      100.00      100.00      100.00      100.00      100.00      100.00      100.00
## y      96.44      96.47      96.47      96.47      96.47      96.47      96.48
##      107 comps
## X      100.00
## y      96.58
```

- (b) Plot the obtained prediction errors from cross-validation, see lecture notes. How many components seem to be optimal? What is the resulting RMSE?

```
# find optimal number of components based on one sigma
cv_results <- RMSEP(model.pcr, estimate = "CV")
comp_opt <- selectNcomp(model.pcr, method = "onesigma", plot = TRUE)
```

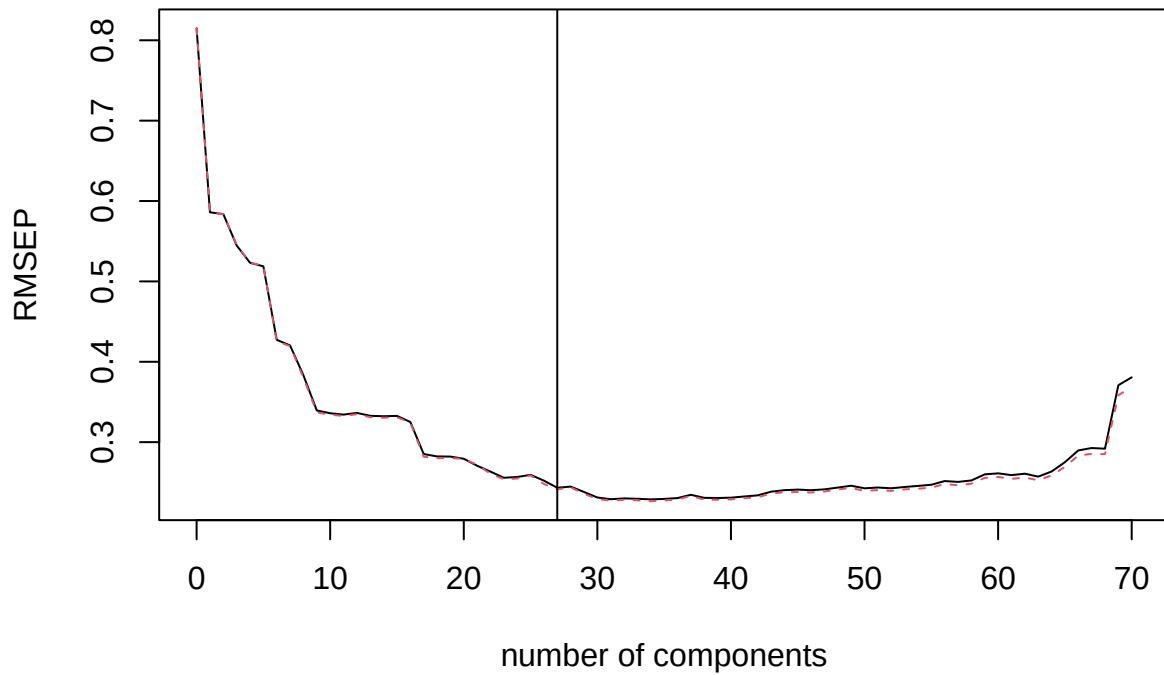


```
paste("The optimal number of components is",
      comp_opt,
      "with an RMSE of",
      cv_results$val[1,,comp_opt])
```

```
## [1] "The optimal number of components is 27 with an RMSE of 0.252136946433096"
```

```
# zoomed in plot
validationplot(model.pcr, val.type = "RMSEP", ncomp = 1:70,
               main = "Zoomed in Validation Plot")
abline(v = comp_opt)
```

Zoomed in Validation Plot

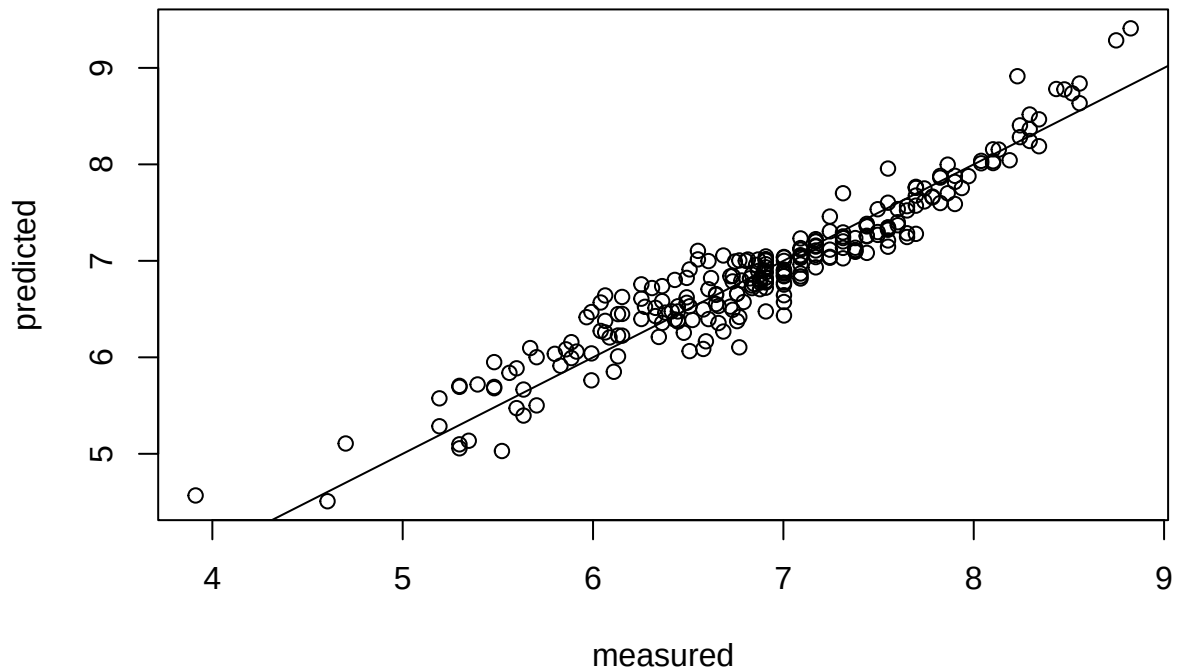


Based on lecture discussions, the optimal number of components can be found by determining the minimum RMSE, then selecting the number of components below that which achieved the minimum RMSE while being within one standard error of this value.

- (c) Use the function `predplot()` to plot the measured y values against the cross-validated y values considering the optimal model.

```
predplot(model.pcr, comp_opt, line=TRUE)
```

y, 27 comps, validation



- (d) Plot the predicted versus observed values for the test data, and compute the RMSE. Compare with the values from last exercise.

```
library(Metrics)

test_pcr_pred <- predict(model.pcr, test, comp_opt)
test_pcr_rmse <- round(rmse(test$y, test_pcr_pred), 3)
paste("Test RMSE for",
      comp_opt,
      "Comps:",
      test_pcr_rmse)
```

```
## [1] "Test RMSE for 27 Comps: 0.357"
```

The forward selection model was the best performing model from the last exercise, which achieved a test RMSE of 0.236. This model also performed better than the optimal pcr model that resulted in a test RMSE of 0.357.

The forward model used only 16 variables out of the 107 available. PCR selects components that have the largest variance while variable selection looks directly at the relationship between a predictor and the response. For this dataset and train/test split, it happens that the choosing high variance components led to a model with less predictive power than simply directly looking at the relationship between predictors and the response.

- (e) The output object of `pcr()` contains the list elements `$scores` and `$loadings`, which are the matrices $\mathbf{Z}$ and $\mathbf{V}$ from the course notes. Visualize in scatterplots the first two score vectors as well as the first two loadings vectors and try to explain shortly what you see.

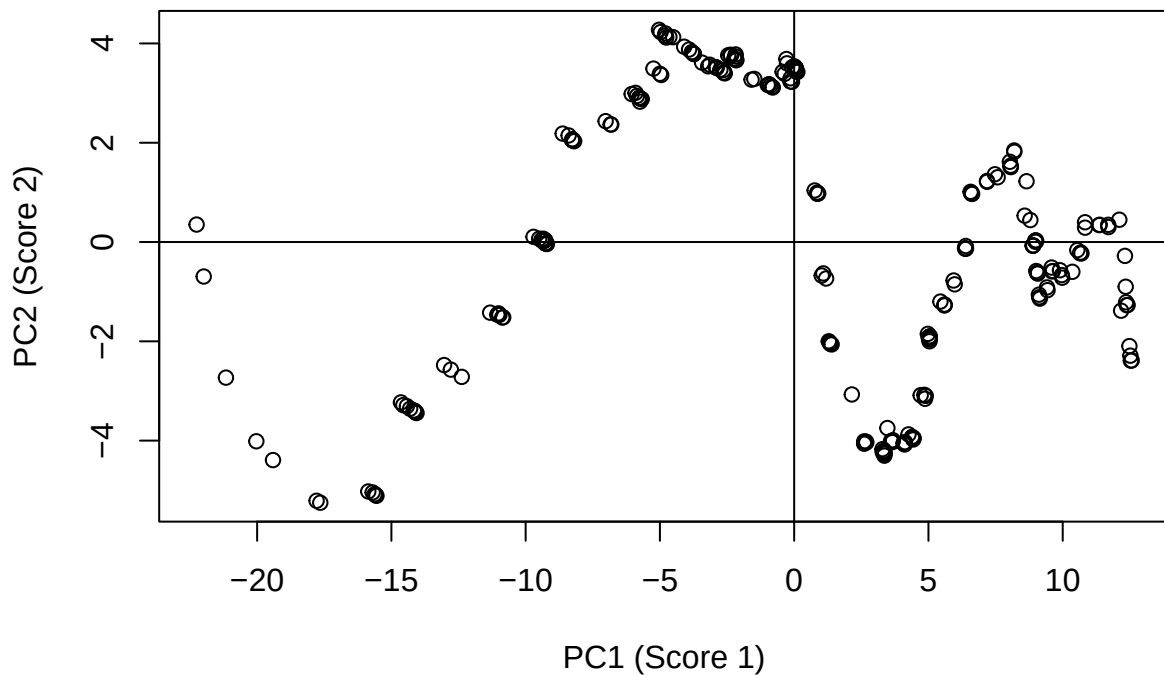
```
# Plot PC1 vs PC2 scores
scores <- model.pcr$scores
plot(scores[, 1], scores[, 2],
      xlab = "PC1 (Score 1)",
```

```

ylab = "PC2 (Score 2)",
main = "Scores Plot")
abline(v=0)
abline(h=0)

```

Scores Plot



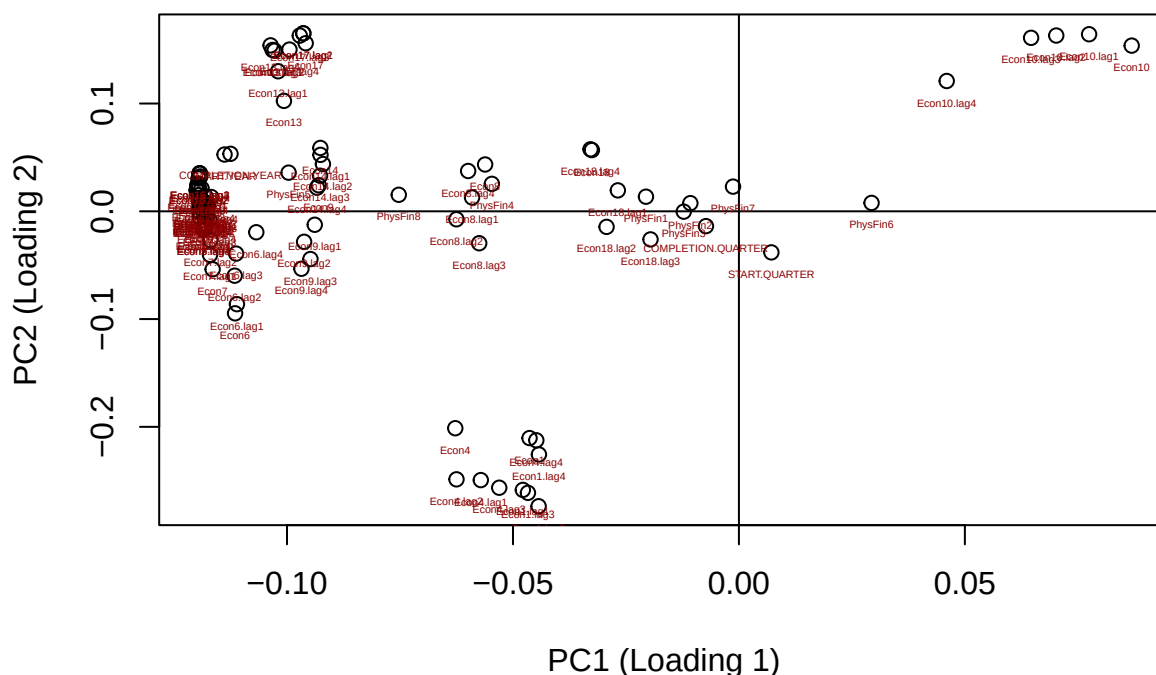
```

# Plot PC1 vs PC2 loadings
loadings <- model.pcr$loadings
plot(loadings[, 1], loadings[, 2],
      xlab = "PC1 (Loading 1)",
      ylab = "PC2 (Loading 2)",
      main = "Loadings Plot")
abline(v=0)
abline(h=0)

text(loadings[, 1], loadings[, 2],
      labels = rownames(loadings),
      pos = 1, cex = 0.3, col = "darkred")

```

Loadings Plot



First looking at the plot of scores, observations that cluster together are similar in the original high-dimensional space. The plot shows many clusters with a few points each. Also, the plot shows outliers such as the points along the lower left quadrant, which have large absolute values along both axes.

The loadings plot shows how variables contribute to the principal components, with variables farther away from the origin having larger contributions. Additionally, variables that cluster together are highly correlated. The plot shows many variables that are highly correlated around $(-0.12, 0.0)$. In general, variables that are lagged versions of another (e.g. `Econ10.lag1`, `Econ10.lag2`, etc.) are similar.

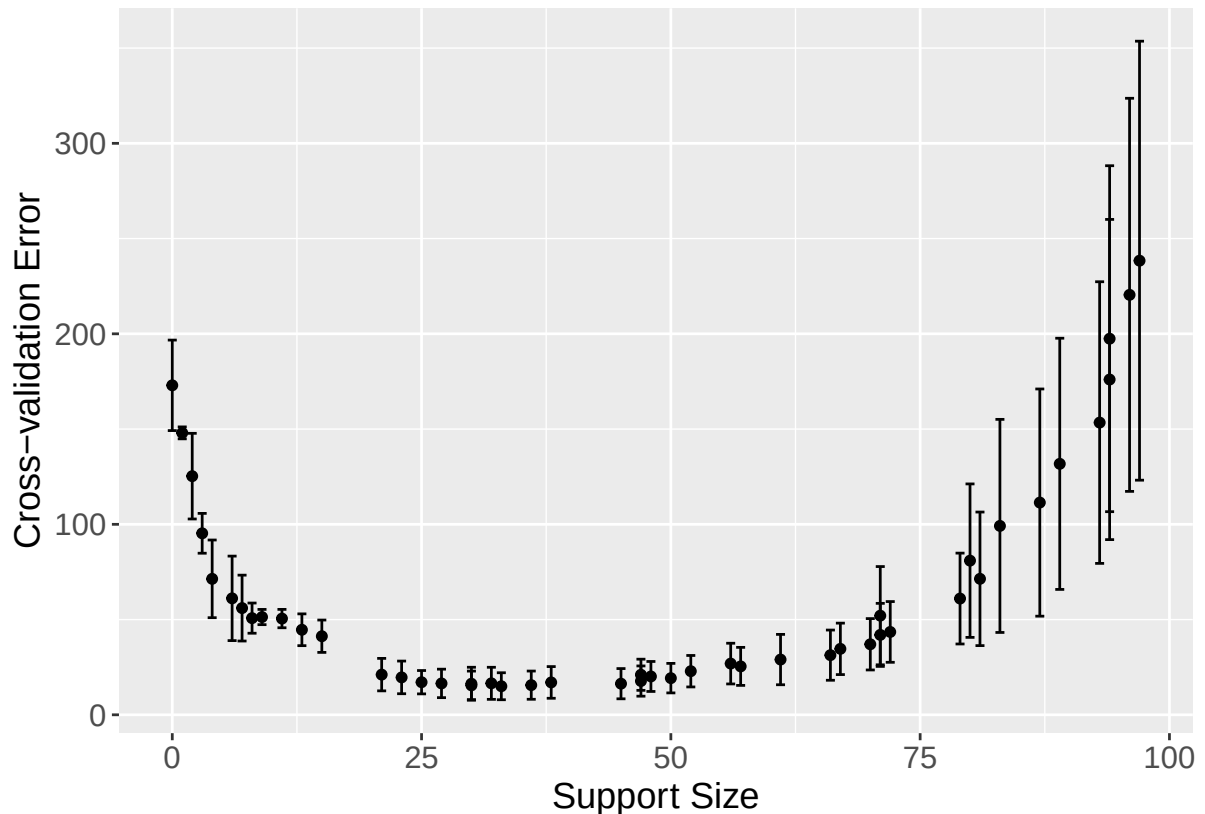
(2) l_0 -norm regression:

Take a quick look at Equation (1) in <https://arxiv.org/pdf/2202.04820>. This l_0 norm regularization has the effect that the size of the tuning parameter λ determines the number of variables included in the model. Thus, this is a convenient way to do variable selection. We only need to identify an appropriate parameter λ . This choice can be made with the function `L0Learn.cvfit` of the package `L0Learn`. You need to provide $\mathbf{X}$ as matrix (`as.matrix()`), the response, and you can set a maximum size of the model (parameter `maxSuppSize`).

- (a) Use the training data to fit the model. Use `plot()` on the result object. How can you interpret the plot? How can you select the optimal model?

```
library(L0Learn)
```

```
L0.model <- L0Learn.cvfit(as.matrix(subset(train, select = -c(y))), train$y)
plot(L0.model)
```

The plot shows the mean cross-validation error model sizes ranging from 0 to the `maxSuppSize`. Though `cvfit` does not create models of every possible size between those two values. For each point, the mean error and standard deviation is shown.

The optimal model can be selected by first looking for the one with the smallest mean error. Then, choosing the smallest possible model that is still within one standard error of the smallest mean error.

- (b) Which is the optimal tuning parameter λ . Extract the regression coefficients for the optimal λ . Which variables are in the model? *Hint*: See help of the function.

```
# find minimum error index and its sd
min_index <- which.min(unlist(L0.model$cvMeans[,]))
min_RMSE <- L0.model$cvMeans[[1]][min_index]
min_SD <- L0.model$cvSDs[[1]][min_index]

# find index of optimal
rmse_values <- L0.model$cvMeans[[1]][1:min_index] # trim RMSEs after min_index
rmse_values <- rmse_values - min_RMSE - min_SD # find largest negative val
opt_index <- 13 # by inspection of rmse_values

# find optimal lambda
lambda_opt <- L0.model$fit$lambda[[1]][opt_index]
paste("The optimal tuning parameter is", lambda_opt)

## [1] "The optimal tuning parameter is 6.49837e-05"

# extract coeffs and vars from optimal model
sparse_matrix <- coef(L0.model, lambda = lambda_opt)

# convert sparse matrix to vector
```

```

beta_vector <- as.vector(sparse_matrix)

# get coefs and their indices
nonzero_indices <- which(beta_vector != 0)
variable_indices <- nonzero_indices[nonzero_indices > 1] - 1 # shift because of intercept
l0_optimal_coefs <- beta_vector[nonzero_indices][-1] # remove intercept

# Create df
l0_optimal_coefs_df <- data.frame(
  variable = colnames(df)[-1][variable_indices], # remove y
  coefficient = l0_optimal_coefs
)
l0_optimal_coefs_df

##      variable      coefficient
## 1   PhysFin1 -2.289803e-02
## 2   PhysFin2  7.471064e-05
## 3   PhysFin4 -3.982326e-04
## 4   PhysFin5 -2.210264e-03
## 5   PhysFin6  7.082536e-04
## 6   PhysFin7  2.369177e-02
## 7   PhysFin8  5.566711e-04
## 8      Econ2  1.362400e-03
## 9      Econ8  4.437296e-04
## 10     Econ10 4.272797e-02
## 11     Econ17 3.223158e-05
## 12     Econ18 -2.367201e-06
## 13 Econ4.lag1 -1.710108e-02
## 14 Econ9.lag1  4.670207e-06
## 15 Econ13.lag1 -1.592303e-05
## 16 Econ8.lag2  4.501815e-04
## 17 Econ9.lag3  4.059507e-06
## 18 Econ14.lag3 8.964183e-05
## 19 Econ1.lag4  5.839546e-05
## 20 Econ10.lag4 1.273389e-01
## 21 Econ12.lag4 2.710939e-04

```

- (c) Plot the fitted values against the response for the training data and compute the RMSE. Compare to PCR.

```

# l0
train_l0_pred <- predict(L0.model,
  newx = as.matrix(subset(train, select = -c(y))),
  lambda = lambda_opt)
train_l0_rmse <- round(rmse(train$y, as.vector(train_l0_pred)), 3)

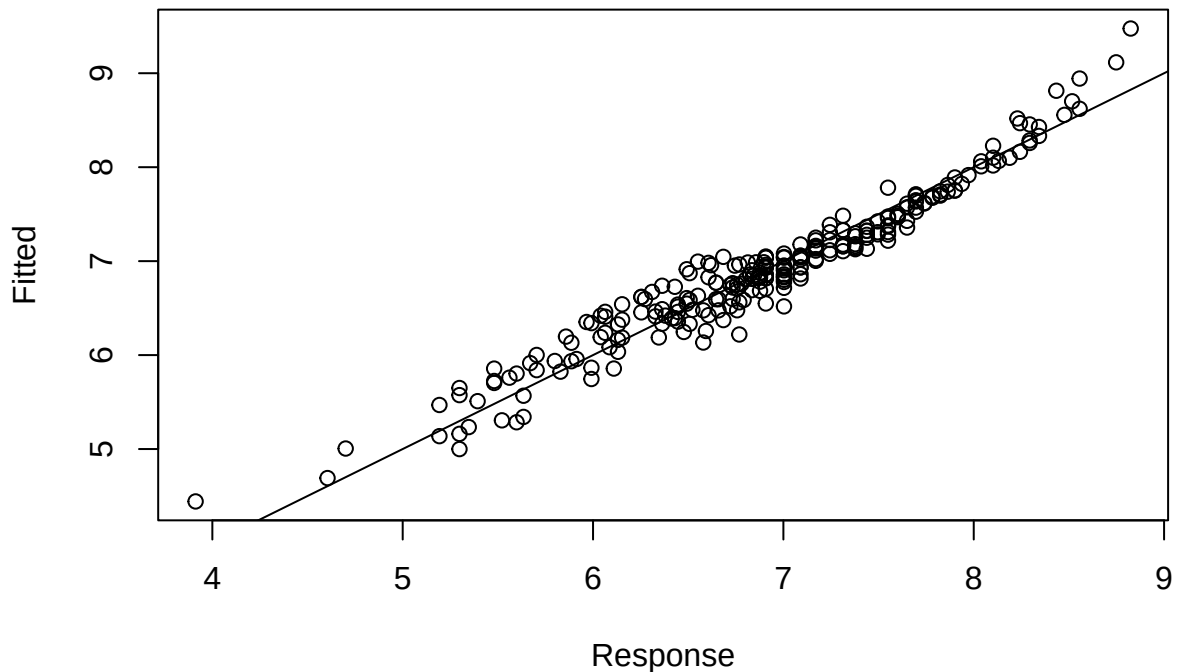
# PCR
train_pcr_pred <- predict(model.pcr, train, comp_opt)
train_pcr_rmse <- round(rmse(train$y, train_pcr_pred), 3)

# l0 plotting
plot(train$y, train_l0_pred,
  xlab = "Response",
  ylab = "Fitted",
  main = "l0 Fitted vs Response for Train"
)

```

```
)
abline(a = 0, b = 1)
```

l0 Fitted vs Response for Train



```
# output
paste("PCR train RMSE:", train_pcr_rmse)
```

```
## [1] "PCR train RMSE: 0.211"
```

```
paste("l0 train RMSE:", train_l0_rmse)
```

```
## [1] "l0 train RMSE: 0.198"
```

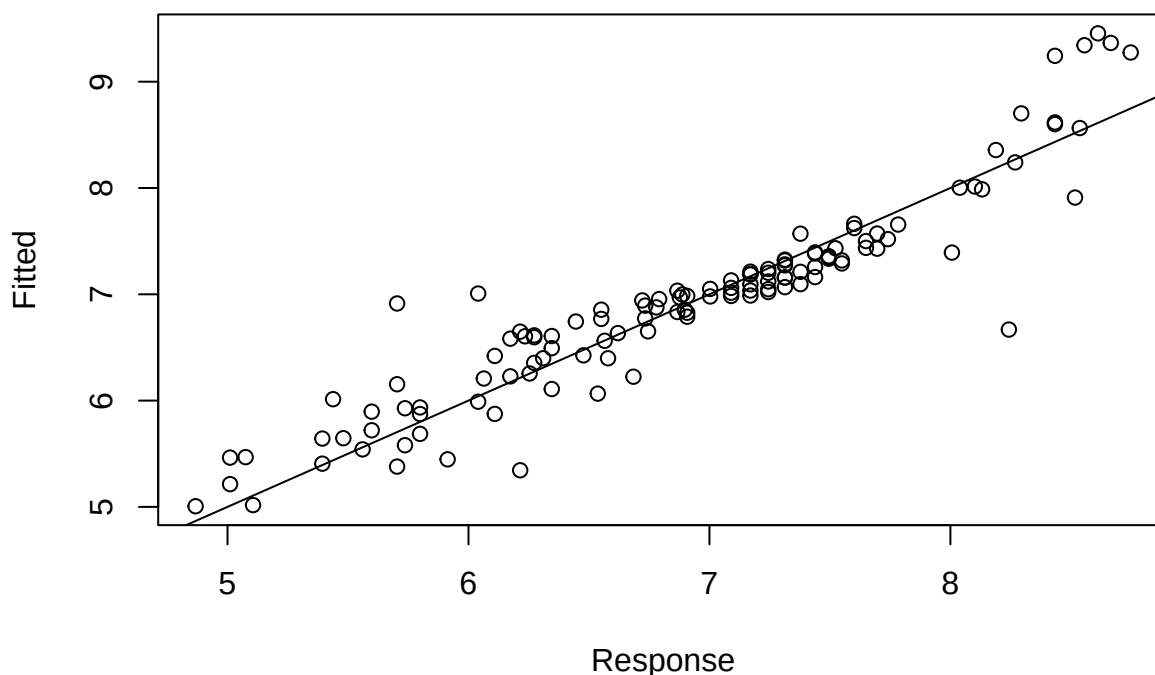
Both models performed similarly on the train set, though the l_0 model was slightly better.

(d) Plot the fitted values against the response for the test data and compute the RMSE. Draw conclusions.

```
# l0
test_l0_pred <- predict(L0.model,
                        newx = as.matrix(subset(test, select = -c(y))),
                        lambda = lambda_opt)
test_l0_rmse <- round(rmse(test$y, as.vector(test_l0_pred)), 3)
```

```
# l0 plotting
plot(test$y, test_l0_pred,
     xlab = "Response",
     ylab = "Fitted",
     main = "l0 Fitted vs Response for Test"
)
abline(a = 0, b = 1)
```

I0 Fitted vs Response for Test



```
# PCR vs l0 comparison
paste("PCR test RMSE:", test_pcr_rmse)
```

```
## [1] "PCR test RMSE: 0.357"
```

```
paste("L0 test RMSE:", test_l0_rmse)
```

```
## [1] "L0 test RMSE: 0.335"
```

Both models were again fairly similar in performance, with the l_0 model being slightly better. The forward selection model in the previous exercise was shown to be better than PCR, and l_0 functions similarly in that it explicitly selects predictors based on their relationship with the response. Since both the forward selection and l_0 models contained a small fraction of the predictors, the predictive power of PCR may have been watered down since the principal components incorporates all variables. These reasons could then explain why l_0 outperformed PCR.

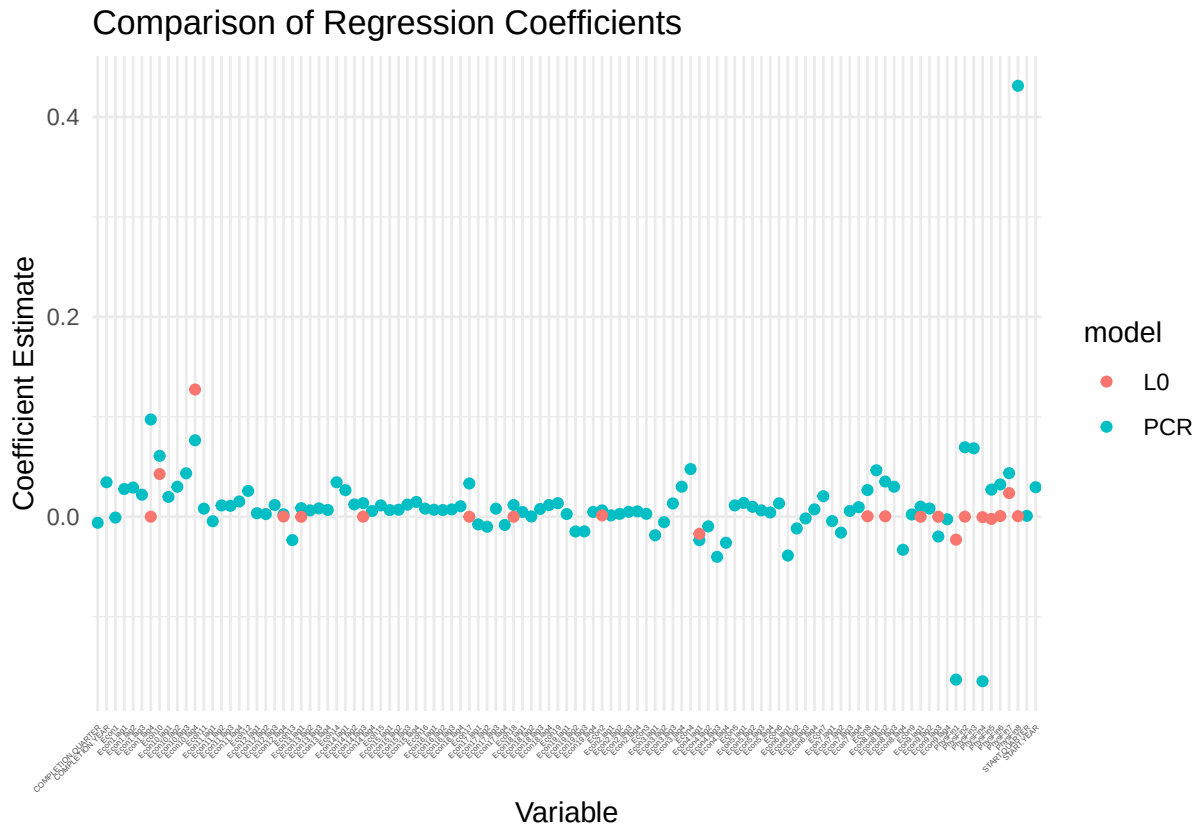
- (e) Compare the regression coefficients (without intercept) from PCR and l_0 -norm regression in a plot and draw conclusions.

```
# load optimal pcr coefs into df
pcr_optimal_coefs <- model.pcr$coefficients[, "27 comps"]
pcr_optimal_coefs_df <- data.frame(
  variable = names(pcr_optimal_coefs),
  coefficient = unname(pcr_optimal_coefs)
)

# combine dfs
pcr_optimal_coefs_df$model <- "PCR"
l0_optimal_coefs_df$model <- "L0"
coef_df <- rbind(pcr_optimal_coefs_df, l0_optimal_coefs_df)
```

```
library(ggplot2)

ggplot(coef_df, aes(x = variable, y = coefficient, fill = model)) +
  geom_point(aes(color = model)) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1, size = 3)) +
  labs(title = "Comparison of Regression Coefficients",
       x = "Variable",
       y = "Coefficient Estimate")
```



In general, l_0 is sparse compared to PCR. This shows that only a few predictors are relevant in estimating the response. The PCR model has many variables where the coefficients were not driven down to zero compared to l_0 . This indicates that the model may be capturing noise or using irrelevant predictors. There were a few variables where PCR estimated especially large coefficients compared to l_0 such as `PhysFin1`, `PhyFin4`, and `PhysFin8`. These variables could have outsized contributions to the variance compared to the predictive power.